DGSI Lab 6 Report - Two Apps Talking

3D Printer Supply Chain Simulator

2026-06-08

Lab 6 Report - Two Apps Talking

A. System Architecture

Lab 6 moved the Week 5 factory simulator from a single-app production planner into the first distributed version of the supply chain. The goal was deliberately narrow: build a provider app, keep it as an independent process with its own database, and teach the manufacturer to buy raw parts through a REST contract instead of reaching into the provider's state.

```
flowchart LR
    M[Manufacturer app<br/>FastAPI + CLI<br/>:8002]
    P[Provider app<br/>FastAPI + CLI<br/>:8001]
    MDB[(manufacturer.db)]
    PDB[(provider.db)]

    M -->|GET /api/catalog| P
    M -->|POST /api/orders| P
    M -->|GET /api/orders/id| P
    M --> MDB
    P --> PDB
```

The provider is responsible for catalog, pricing tiers, stock, order lifecycle, and event history. The manufacturer remains responsible for its own raw-parts inventory and keeps a local purchase order record for every remote provider order. This duplication is intentional: the provider owns the seller's view of the transaction, while the manufacturer owns the buyer's view and needs to know what is in flight.

The main design decision was to use separate FastAPI apps instead of a shared Python module. That made the boundary explicit and forced the same integration pattern that real companies use: request, response, status polling, and error handling. The cost is more operational work because both services must be running and their simulated calendars must stay aligned.

Provider REST Contract

The provider contract was designed around the minimum actions the manufacturer needs:

Method	Endpoint	Reason
`GET`	`/api/catalog`	Manufacturer can discover product IDs, lead times, stock, and price tiers.
`GET`	`/api/stock`	Humans and agents can inspect available supplier capacity.
`POST`	`/api/orders`	Manufacturer places a parts order with buyer, product, and quantity.
`GET`	`/api/orders`	Used for status lists and debugging.
`GET`	`/api/orders/{id}`	Manufacturer polls one remote order until delivered.
`POST`	`/api/day/advance`	Provider processes transitions for one simulated day.
`GET`	`/api/day/current`	Turn alignment check.
`GET`	`/api/events`	Audit trail and debugging evidence.

When an order is created, the provider checks stock immediately, picks the correct tier price, subtracts the quantity from its own stock, computes `expected_delivery_day`, and writes `order_placed` plus `stock_updated` events. The lifecycle then advances through `pending`, `confirmed`, `in_progress`, `shipped`, and `delivered` during provider day advancement.

Manufacturer Changes

On the manufacturer side, the important addition was `ProviderClient`, which wraps HTTP calls to the provider. The manufacturer now has provider config in `manufacturer/config.json`, CLI commands such as `suppliers catalog`, `purchase create`, and `purchase list`, and local purchase order rows that store the remote provider order ID. During `manufacturer/app/services.py::advance_day`, open purchases are polled from the provider and delivered parts are added to local inventory.

This polling design was chosen over provider push notifications. Polling is less elegant but much simpler for a lab environment: it avoids callbacks, firewalls, retry

queues, and cross-service subscriptions. For a day-based simulation, polling once per day is also semantically clear.

B. Agent Design

Lab 6 did not require autonomous agents yet. The important agent-related design work was to make the future agent interface simple and consistent. Both apps expose CLIs, and the commands use the same nouns that appear in the REST API:

```
./provider/start_cli.sh catalog
./provider/start_cli.sh stock
./provider/start_cli.sh orders list
./provider/start_cli.sh day advance

./manufacturer/start_cli.sh suppliers list
./manufacturer/start_cli.sh suppliers catalog "ChipSupply Co"
./manufacturer/start_cli.sh purchase create --supplier "ChipSupply Co" --product pcb --qty 5
./manufacturer/start_cli.sh purchase list
```

That CLI consistency matters because Week 7 and Week 8 role skills later depend on exact commands. The Lab 6 implementation therefore created the operational vocabulary that the agents would use: catalog inspection, stock inspection, purchase creation, order status checks, and day advancement.

The main constraint established here was also the most important later rule: role actors must not advance time on their own. The simulation day should be advanced by a human in Lab 6 and by a turn engine later.

C. Simulation Results

The Lab 6 verification scenario was the five-day provider/manufacturer flow:

1. Start provider on port `8001` and manufacturer on port `8002`.
2. Confirm that the provider has stock and a tiered catalog.
3. Place a manufacturer purchase order for provider parts.
4. Advance provider and manufacturer in the same order each day.
5. Confirm the order becomes delivered remotely and the manufacturer receives inventory.

The repository supports this scenario through the app CLIs and the smoke script:

```
./scripts/check_supply_chain.sh
```

The most important behavior is not that the order exists, but that both databases tell the same story from their own side. The provider records the order as seller history and decrements supplier stock. The manufacturer records an inbound purchase order and only increases local inventory after polling a

delivered provider order. This avoids the common distributed-system mistake of assuming that a successful remote order means the materials have already arrived.

The lead-time rule is also enforced: parts ordered today do not arrive today. The provider computes delivery as current day plus effective lead time, with a minimum of one day. This is what creates planning pressure for later labs.

D. Vibe-Coding Reflection

Claude Code was most useful in Lab 6 for generating the repeated scaffolding: FastAPI endpoints, Typer CLI wrappers, SQLAlchemy models, and serialization methods. The pattern was clear enough that the agent could extend it without needing major architectural invention.

The parts that needed human correction were the distributed-system boundaries. Early suggestions tended to blur responsibility by sharing logic too directly or by making the manufacturer assume provider state. We corrected that by keeping databases separate and forcing all cross-app communication through HTTP.

The other lesson was that event logs are not optional. Without provider and manufacturer events, a failed order looks like a black box. With events, we can reconstruct whether the order failed at stock check, shipment, polling, or local receipt. That audit trail became essential in Week 8 when many more orders and agents were active.

If we restarted Lab 6, the main improvement would be to write an automated five-day scenario test earlier. Manual CLI verification is fine for the first pass, but the moment the retailer and engine were added, repeatable smoke checks became more valuable than ad hoc terminal history.

DGSI Lab 7 Report - Retailer, Turn Engine, and First Skill

3D Printer Supply Chain Simulator

2026-06-08

Lab 7 Report - Retailer, Turn Engine, and First Skill

A. System Architecture

Lab 7 completed the three-app supply chain and introduced orchestration. The new app was the retailer, which receives customer demand, fulfills from stock, backorders when necessary, and buys finished printers from the manufacturer. The new script was the turn engine, which advances all services in lock-step instead of relying on a human to type commands in three terminals.

```
sequenceDiagram
    participant TE as Turn Engine
    participant R as Retailer :8003
    participant M as Manufacturer :8002
    participant P as Provider :8001

    TE->>R: POST /api/orders customer demand
    TE->>R: deterministic retailer turn
    R->>M: POST /api/orders printer purchase
    TE->>M: deterministic manufacturer turn
    M->>P: POST /api/orders parts purchase
    TE->>P: provider turn
    TE->>R: POST /api/day/advance
    TE->>M: POST /api/day/advance
    TE->>P: POST /api/day/advance
```

The architecture now has a downstream demand source and a complete upstream reaction path:

- Customer orders pressure the retailer.
- Retailer purchase orders pressure the manufacturer.
- Manufacturer part orders pressure the provider.

- Provider lead times and stock limits constrain the whole chain.

The retailer is intentionally built like the other apps: FastAPI, Typer CLI, SQLite, seed data, event log, day counter, and JSON import/export. Its key endpoints are `/api/catalog`, `/api/stock`, `/api/orders`, `/api/purchases`, and `/api/day/advance`. The retailer also has `/api/catalog/sync`, which pulls wholesale prices from the manufacturer so retail price rules can enforce a markup over the current upstream price.

Turn Engine Design

The deterministic Week 7 engine lives in `scripts/turn_engine.py`. It reads `scenarios/week7.json`, validates the current day, generates customer orders from a demand model, runs simple role policies, advances apps, and writes a JSONL record to `logs/turn-engine.jsonl`.

The chosen turn order is downstream pressure first, then upstream reaction:

1. Generate customer orders at the retailer.
2. Retailer processes pending customer orders and reorders printers if stock is below target.
3. Manufacturer releases pending sales orders and orders missing BOM parts.
4. Provider observes open orders.
5. All apps advance one day.

This order was chosen because it creates a clear causal chain. Demand appears at the customer edge first. The retailer reacts by ordering printers. The manufacturer reacts by releasing production and buying parts. The provider then handles its own order lifecycle. The day advance is centralized so no role can accidentally move time twice.

B. Agent Design

The first skill file was `skills/manufacturer-manager.md`. The manufacturer was the best first role because it touches both directions of the chain: retailer orders downstream and provider purchases upstream.

The skill's core responsibilities are:

- Review incoming retailer orders.
- Check raw material and finished-printer stock.
- Release sales orders to production when possible.
- Order parts from suppliers when stock or projected demand requires it.
- Adjust wholesale prices when utilization and backlog become high.
- Never call `day advance`.

Two skill-authoring decisions mattered most:

1. **The skill names exact commands.** The agent is not asked to invent a command vocabulary. It gets commands such as

`./manufacturer/start_cli.sh sales orders`, `production release`, `purchase create`, and `price set`.

2. **The skill forbids time advancement.** This prevents the classic orchestration failure where an enthusiastic role agent finishes its work and calls `day advance`, causing the global simulation to skip or desynchronize.

The later Week 8 skill is shorter than the initial Lab 7 draft, but the Lab 7 lesson remained: the agent needs a clear role, a command surface, constraints, and a required summary. Without the summary, the raw output is difficult to audit. With it, the engine can store one file per role per day and the team can inspect what happened.

C. Simulation Results

The Lab 7 baseline scenario is `scenarios/week7.json`. It defines three days of demand with normal, bump, and quiet signals. The engine can be run with:

```
python3 scripts/turn_engine.py --scenario scenarios/week7.json --days 3
```

The expected result is not a polished autonomous economy yet. The goal is plumbing:

- Customer demand is inserted into the retailer.
- Retailer orders are visible in the manufacturer.
- Manufacturer can release production and create provider purchases.
- Provider receives parts orders.
- All apps move to the next simulated day together.
- Events across the three services form a coherent trace.

The repo also contains agent-check logs in `logs/skill-manufacturer-check.log`, `logs/skill-provider-check.log`, and `logs/skill-retailer-check.log`. Those files are useful evidence that the skill format was tested in isolation before the full Week 8 run.

The most useful failure mode in Lab 7 was command ambiguity. If a skill said "review stock and buy what is needed," the model could spend time listing data or choose the wrong command. When the skill specified exact commands and a fixed decision order, output became more predictable. This is why the final skill files evolved toward fast-path actions and explicit end-of-day summaries.

D. Vibe-Coding Reflection

Claude Code was strong at repeating the established service pattern from Lab 6. Once provider and manufacturer had clear FastAPI/CLI/service/database layers, the retailer could be scaffolded quickly. The AI also helped create the initial turn engine skeleton because the orchestration algorithm was easy to express in natural language.

The weaker area was multi-process reasoning. A local function call and a remote

HTTP call can look similar in code, but they have very different failure modes. The team had to keep asking: which app owns this state, which API is the contract, and what happens if the upstream service is down?

The PRD-first approach helped because it prevented the retailer from becoming just another table inside the manufacturer. We had already committed to separate processes and event logs, so when the third app arrived, the architecture had a place for it.

If we started Lab 7 again, we would add log capture from the first engine run immediately. The statement explicitly says agent output should be stored, not just printed, and the later analysis work proved why: once a run has 15 or 25 days, terminal scrollback is not evidence.

The main successful outcome of Lab 7 was that autonomy had a stable place to attach. By the end, the software was no longer only three services; it was a world with a clock, customer demand, role turns, state transitions, and an audit trail.

DGSI Lab 8 Report - Autonomous Supply Chain and Analysis

3D Printer Supply Chain Simulator

2026-06-08

Lab 8 Report - Autonomous Supply Chain and Analysis

A. System Architecture

Lab 8 completed the three-week arc: all three roles have skill files, the scenarios include real market pressure, and the simulation produces logs and charts that can be analyzed after the run. The final system keeps execution deterministic where correctness matters and uses role skills where business decisions are needed.

```
flowchart LR
    subgraph World[Shared simulated world]
      C[Customer demand]
      R[Retailer<br/>orders, stock, prices]
      M[Manufacturer<br/>BOM, production, stock]
      P[Provider<br/>parts, tiers, lead times]
    end

    E[Extended runner<br/>scripts/run_simulation.py]
    S[Scenario JSON<br/>demand/supply/lead-time signals]
    K[Skill files<br/>provider/manufacturer/retailer]
    L[Per-day logs + metrics]
    G[Matplotlib charts]

    S --> E
    K --> E
    E --> C
    C --> R
    R -->|printer orders| M
    M -->|parts orders| P
    P -->|deliveries polled| M
```

```
M -->|deliveries polled| R
E --> L
L --> G
```

The final runner is `scripts/run_simulation.py`. It reads a scenario, applies the day signal, injects customer orders, runs the three role turns, advances all apps, and writes metrics. The scenario merge rule for overlapping events is multiplicative. For example, during the overlap of chip shortage and Christmas rush, demand and lead-time modifiers compound instead of replacing one another. This makes days 18-20 of `holiday-rush` the strongest stress period.

The service boundaries stayed the same as Lab 7:

- Provider owns part stock, tier prices, lead times, and provider orders.
- Manufacturer owns raw inventory, printer models, BOMs, production, finished stock, and retailer sales orders.
- Retailer owns customer demand, retail stock, backorders, purchases from manufacturer, and retail prices.

The final PRD in `docs/PRD.md` now reflects this system rather than the original Week 5 single-factory app.

Data Model

```
erDiagram
    PROVIDER_PRODUCT ||--o{ PROVIDER_PRICING_TIER : has
    PROVIDER_PRODUCT ||--|| PROVIDER_STOCK : stocked_as
    PROVIDER_PRODUCT ||--o{ PROVIDER_ORDER : ordered_as
    PROVIDER_ORDER ||--o{ PROVIDER_EVENT : logs

    MANUFACTURER_PRINTER_MODEL ||--o{ MANUFACTURER_SALES_ORDER : ordered_as
    MANUFACTURER_PRINTER_MODEL ||--|| MANUFACTURER_FINISHED_STOCK : stocked_as
    MANUFACTURER_PRODUCT ||--|| MANUFACTURER_INVENTORY : stocked_as
    MANUFACTURER_SALES_ORDER ||--o{ MANUFACTURER_EVENT : logs
    MANUFACTURER_PURCHASE_ORDER ||--o{ MANUFACTURER_EVENT : logs

    RETAILER_CATALOG_ITEM ||--o{ RETAILER_CUSTOMER_ORDER : ordered_as
    RETAILER_CATALOG_ITEM ||--|| RETAILER_STOCK : stocked_as
    RETAILER_CUSTOMER_ORDER ||--o{ RETAILER_EVENT : logs
    RETAILER_PURCHASE_ORDER ||--o{ RETAILER_EVENT : logs
```

The apps do not share one universal schema because they represent different organizations. This was a good trade-off: it avoids coupling, but it means the turn engine and reports must normalize metrics across different local models.

B. Agent Design

The final skill set contains:

- `skills/provider-manager.md`
- `skills/manufacturer-manager.md`
- `skills/retail-manager.md`

The provider skill manages restocking and supplier price changes. It watches stock against starting thresholds and reacts to shortage signals with controlled price increases. The manufacturer skill manages order release, part purchasing, and wholesale price increases when backlog or utilization is high. The retail skill processes customer orders, backorders unmet demand, creates printer purchases, and adjusts retail prices while respecting the wholesale markup floor.

The first versions of the skills were too exploratory. They asked the agent to inspect state, reason, choose commands, and act. That worked for one-day demos but became slow and brittle in long simulations. The final skills use a fast path: the runner provides current state plus an `ACTION HINT`, and the role executes recommended commands or reports no action. This preserved the educational idea of role skills while keeping long runs auditable and short enough.

One important rewrite was the provider behavior. The Lab 8 statement explicitly warns that provider stockouts can cascade. The skill therefore gained rules about restocking products below threshold and limiting daily price changes. Another rewrite was the retailer behavior. The retailer has to process every pending order, otherwise the backlog state becomes misleading. The final retail skill starts with `process-orders` and then places purchases sized against backlog and buffer.

What the agents are good at:

- Following explicit command hints.
- Writing readable summaries of why they acted.
- Applying simple threshold rules.
- Keeping local decisions separated by role.

What they are bad at:

- Discovering the right command in a large CLI surface without help.
- Planning several lead times ahead unless the prompt makes that look-ahead explicit.
- Balancing price, service level, and stock without turning the rule into a concrete action.

C. Simulation Results

Two scenarios were used for analysis:

- `calm-market`: stable control run, 15 logged days.
- `holiday-rush`: volatile run, 25 logged days with normal demand, Black Friday, chip shortage, and Christmas rush.

The generated evidence is stored as metrics logs in `logs/` and charts in `reports/`.

The dashboard summarizes the full 25-day holiday-rush run across all three roles. Inventory, prices, and order fulfillment all show the signature of a stressed supply chain.

Inventory starts under pressure and never fully recovers at the retail level. Retail stock begins at 8 units and repeatedly hits zero. Manufacturer raw stock dips to a minimum of 57 units during the stressed middle days, then recovers to 295 by day 25 as provider deliveries catch up. This late recovery is delayed, not anticipatory.

Prices rise persistently across all roles. Retail prices for the three printer models reach `1100.14`, `780.04`, and `1936.86` by day 25. Wholesale prices follow with values of `546.67`, `367.73`, and `957.79`. Provider tier prices increase on constrained parts — `kit_piezas` tier 10 rises from `135.0` to `171.52`, and `transformador_24v` tier 50 from `18.0` to `23.96`. The movement is a sustained escalation in response to backlog, not oscillation.

Orders tell a similar story. The run receives 212 customer orders over 25 days, fulfills 164, and ends with 48 backordered. The largest demand peak is day 20, with a catch-up burst of 38 fulfilled orders driven by stock that had accumulated from earlier upstream activity.

For more detail on each role individually, the per-role breakdown charts are available in the `reports/` folder: `holiday-rush_manufacturer_detail.png`, `holiday-rush_provider_detail.png`, and `holiday-rush_retailer_detail.png`.

The comparison chart puts the calm-market and holiday-rush runs side by side. The calm run is a useful baseline: 80 customer orders over 15 days, 59 fulfilled, with stable prices and gradual stock depletion. Compared to the holiday run, the difference in scale and volatility is immediately visible. Retail prices in the holiday scenario end up roughly double those of the calm run, and the order volume is more than twice as large. The calm run shows that the system is already under-provisioned at baseline — the holiday events do not create a different kind of failure, they amplify the same structural weakness.

Event Overlay and Causal Chain

The event phases explain the shape of the volatile run:

- Days 1-7: normal demand still drains retailer stock because starting inventory is small.
- Days 8-12: Black Friday increases demand before enough finished printers can arrive.
- Days 13-20: chip shortage reduces supplier effectiveness and extends lead times.

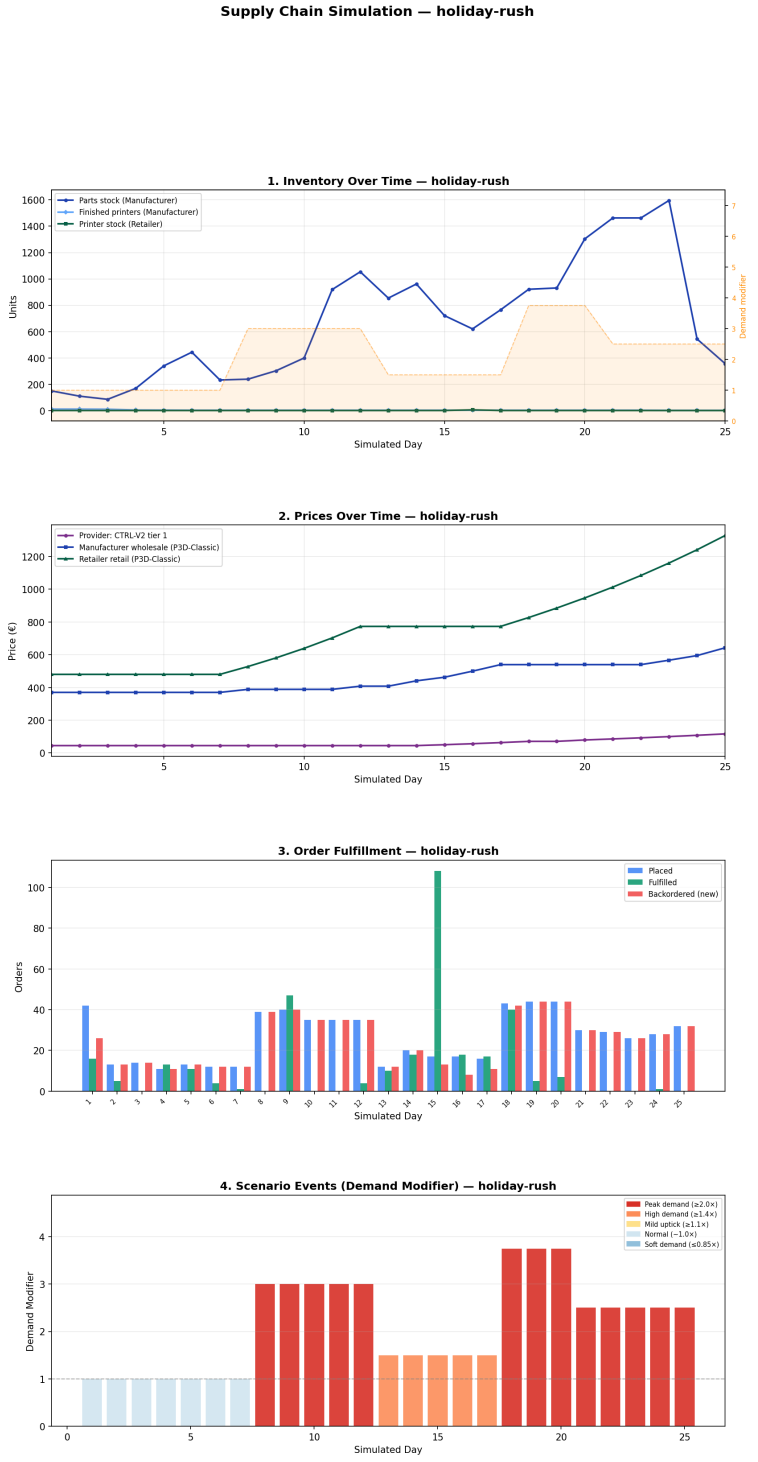

Figure 1: Holiday rush dashboard

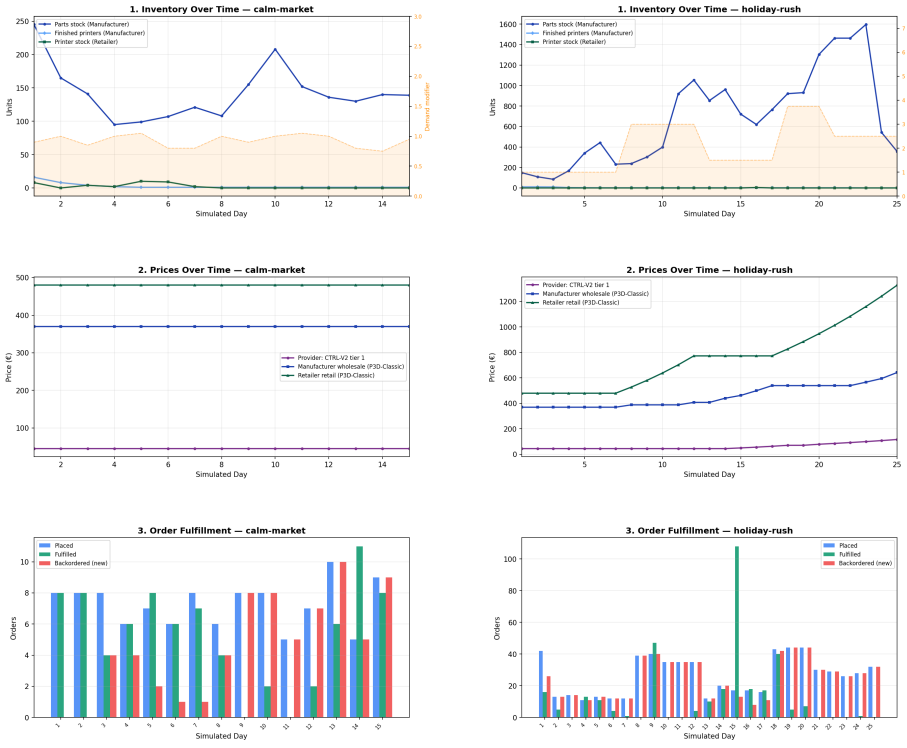

Figure 2: Scenario comparison

- Days 18-20: chip shortage and Christmas overlap, multiplying the stress.
- Days 21-25: Christmas demand remains high, but prices and recovered upstream stock dampen the later collapse.

The most visible emergent behavior is a bullwhip pattern, unfolding in three compounding phases. During days 1–7, both the retailer and the manufacturer operate under normal demand assumptions. Neither agent accumulates a sufficiently large safety stock: the retailer keeps minimal printer inventory, and the manufacturer does not pre-build finished units in anticipation of a surge. When Black Friday hits on day 8, demand spikes abruptly and neither agent is prepared.

This triggers a reactive scramble. The retailer, facing stockouts, places aggressively large orders on the manufacturer. The manufacturer, in turn, scales up production and issues oversized part orders to the provider, each printer requiring multiple components, so even a moderate increase in production targets translates into a disproportionately large upstream parts order, further inflated by safety-stock logic trying to compensate for the accumulated backlog. The upstream signal is already a distorted, magnified version of the original retail demand shock.

Then, on day 13, the chip shortage hits. Precisely when the manufacturer is placing its largest part orders, supplier lead times extend and delivery rates drop. The incoming parts flow slows just as production pressure peaks, making it impossible for the manufacturer to clear the backlog through output. The two stressors (demand surge and supply disruption) overlap and multiply: the retailer's stockouts persist not just because demand is high, but because the upstream pipeline has seized up at the worst possible moment.

By days 18–20, Christmas demand compounds the already unresolved Black Friday backlog. Only after day 21, as the chip shortage eases and previously-placed large orders finally arrive, does the manufacturer's raw-parts stock begin to recover. Critically, this upstream recovery is not a sign of restored equilibrium, it is evidence of order inertia: panic-driven purchases placed during peak stress keep arriving even as downstream demand begins to normalize, setting the stage for a potential overstock correction in subsequent periods.

Required Questions

Did the manufacturer build stock ahead of Black Friday?
Only partially. It reacted to retailer demand and released orders, but the system did not intentionally prebuild enough finished-printer stock before day 8. Retail stock reached zero during the early stress period, so preparation was reactive rather than anticipatory.

When stockouts happened, whose decision was proximate and whose was root cause?
The proximate cause was retailer exposure: starting stock was too low for the

incoming demand and reorder lead times. The root cause was system-wide planning latency: the manufacturer and provider reacted after demand became visible, and the compounded lead times meant upstream correction arrived late.

Did prices stabilize or oscillate?
Prices mostly trended upward. Retail and wholesale prices rose as backlog persisted; provider prices increased on constrained tiers. The run shows damped escalation rather than a price war or collapse.

Can we identify a bullwhip moment?
Yes. The day 8-20 demand shock becomes larger upstream as retailer backorders create printer orders and printer orders expand into BOM part purchases. The later recovery of manufacturer raw stock while backorders still exist is the signature: upstream inventory movement is amplified and delayed relative to the original customer demand.

D. Vibe-Coding Reflection

Across Labs 6-8, Claude Code was most valuable when the architecture was already explicit. It generated repeated service patterns quickly, helped wire APIs and CLIs, and made it easier to evolve the same idea across provider, manufacturer, and retailer.

It struggled when asked to solve a hole big problem at once. "Build the autonomous simulation" was too large. The successful prompts were smaller and more specific: add a provider endpoint, add retailer purchase polling, write a skill that forbids day advance, or summarize a metrics log. The PRD-first workflow helped because each lab had a stable vocabulary and a target architecture before the code changed. One thing we would redesign is using an API key instead of invoking the agent through an external CLI tool. In the current implementation, each agent turn starts a separate Codex/Claude CLI process, passes the skill and current state as a prompt, waits for the tool to execute commands, and then reads the final summary. This approach is convenient for development because it reuses the existing local authentication and tool permissions, but it adds significant overhead in long simulations. A direct API integration would make the simulation faster and easier to control: the runner could send structured prompts directly to the model, receive structured responses, reduce process startup time, and handle retries or errors more consistently.

Another thing we would redesign is observability. We already produce detailed charts when the simulation ends, but the improvement would be to move the charts into the real-time dashboard. At the moment, the analysis charts are generated after the simulation from the CSV logs, which is useful for the final report but less convenient during experimentation. Showing inventory, prices, demand, and backorders live in the Streamlit dashboard would make it much easier to monitor the simulation as it runs, detect failures earlier, and compare how each agent reacts day by day without waiting for the full scenario to finish.

The main lesson is that multi-agent software is less about "smarter prompts" and more about boundaries. The deterministic services execute state changes; the REST contracts keep responsibilities clear; the engine owns time; the skills provide local decision rules; and the logs make the result explainable. When those boundaries hold, the agents can make imperfect decisions without making the system impossible to understand.